

Q.No:3

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n .

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$.

After this, there are m lines describing the teleporters. Each line has two integers a and b : there is a teleporter from the room a to room b .

There are no two teleporters whose starting and ending rooms are the same.

Output

First print an integer k : the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 2 6
```

Test Case 2

INPUT (STDIN)

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
```

EXPECTED OUTPUT

```
1
4
1 2 3 6
```

Mandatory Test Cases

Code:

```
#include <stdio.h>
#include <string.h>

#define MAXN 505

int cap[MAXN][MAXN];
int flow[MAXN][MAXN];
int parent[MAXN];
int visited[MAXN];
```

```
int adj[MAXN][MAXN];
int adjCount[MAXN];
int bfs(int n, int s, int t)
{
    int queue[MAXN];
    int front = 0, rear = 0;

    memset(visited, 0, sizeof(visited));
    memset(parent, -1, sizeof(parent));

    queue[rear++] = s;
    visited[s] = 1;

    while (front < rear)
    {
        int u = queue[front++];

        for (int i = 0; i < adjCount[u]; i++)
        {
            int v = adj[u][i];

            if (!visited[v] && cap[u][v] - flow[u][v] > 0)
            {
                visited[v] = 1;
                parent[v] = u;
                queue[rear++] = v;
            }

            if (v == t)
                return 1;
        }
    }

    return 0;
}

int path[MAXN];
int used[MAXN];

int findPath(int u, int t, int length)
{
    path[length] = u;

    if (u == t)
        return length + 1;

    used[u] = 1;

    for (int i = 0; i < adjCount[u]; i++)
    {
        int v = adj[u][i];
        if (!used[v] && flow[u][v] > 0)
        {
```

```
flow[u][v]--;

int result = findPath(v, t, length + 1);

if (result != -1)
{
    used[u] = 0;
    return result;
}
flow[u][v]++;
}
}
used[u] = 0;
return -1;
}

int main()
{
    int n, m;

    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++)
    {
        int a, b;
        scanf("%d %d", &a, &b);

        if (cap[a][b] == 0)
        {
            adj[a][adjCount[a]++] = b;
            adj[b][adjCount[b]++] = a;
        }

        cap[a][b]++;
    }

    int maxFlow = 0;

    while (bfs(n, 1, n))
    {
        int v = n;

        while (v != 1)
        {
            int u = parent[v];

            flow[u][v]++;
            flow[v][u]--;

            v = u;
        }

        maxFlow++;
    }
}
```

```
}

printf("%d\n", maxFlow);

for (int i = 0; i < maxFlow; i++)
{
    memset(used, 0, sizeof(used));

    int length = findPath(1, n, 0);

    printf("%d\n", length);

    for (int j = 0; j < length; j++)
    {
        if (j > 0)
            printf(" ");

        printf("%d", path[j]);
    }

    printf("\n");
}

return 0;
}
```

Output:

Test_case:1

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
2
3
1 2 6
4
1 3 4 6
```

Test_case:2

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
1
4
1 2 3 6
aadityajaiswal@Aad
```